SBA301 – SLOT 24

CHAPTER 16 – SECURING REST SERVICES WITH SPRING BOOT

PART A: SECURITY CONCEPTS • AUTHENTICATION/AUTHORIZATION • OAUTH 2.0 • JWT

CORE MENTAL MODEL: Tư duy trung tâm của Slot 24: Bảo mật REST không bắt đầu từ việc "thêm một JWT filter". Sinh viên phải hiểu security boundary, danh tính (authentication), quyền truy cập (authorization), credential/token lifecycle, threat model và chuỗi xử lý request qua Spring Security trước khi viết code. JWT là một token format; OAuth 2.0 là một authorization framework — hai khái niệm không đồng nghĩa.

Thông tin	Giá trị
Môn học	SBA301 – Integrate Single Page Application with Spring Boot
Slot	24
Chủ đề	Chapter 16 Part A – Securing REST Services with Spring Boot: security concepts, OAuth/JWT
CLO chính	CLO4 – Apply Spring Boot REST concepts; mở rộng sang bảo vệ REST API và token-based security
Ngữ liệu xuyên suốt	Orchid Management / Lab 06 – Registration, Login, JWT, Protected Endpoints
Course baseline	JDK 21 • Spring Boot 3.x track • Spring Web • Spring Security • Spring Data JPA • SQL Server • Postman
Thời lượng gợi ý	135 phút trên lớp + bài tập/AI verification ngoài giờ
Learning flow	Prepare → Understand → Trace → Implement → Break → Diagnose → Verify → Transfer

NOTE: Scope lock: Slot 24 tập trung vào security concepts, authentication/authorization, Spring Security request lifecycle, OAuth 2.0 roles/flows và JWT fundamentals. Lab-level JWT implementation được dùng để kiểm chứng mental model. Không biến slot này thành khóa học cryptography, không triển khai đầy đủ Authorization Server/OIDC, không đi sâu refresh-token rotation, distributed key management hay enterprise IAM — các nội dung đó được đặt ở production/research extension.

CÁCH SỬ DỤNG GIÁO TRÌNH

Tài liệu giữ cấu trúc của Slot 22 V3: Phần IV là knowledge base lý thuyết mở rộng; Phần VI là implementation workbook. Sinh viên không được chỉ copy JWT code. Mỗi claim về request security flow phải được kiểm bằng source, debugger/log hoặc test; mỗi claim về chuẩn/công nghệ hiện hành phải đối chiếu official documentation.

Pha học	Sinh viên làm gì?	Bằng chứng
Prepare	Chạy baseline REST API; xác nhận DB/user table; ôn HTTP/header/status.	Baseline run + Git checkpoint
Understand	Phân biệt authentication, authorization, session, Basic, OAuth, JWT.	Security concept map
Trace	Theo dõi request qua SecurityFilterChain đến Controller.	Filter-chain execution trace
Implement	Cấu hình security + login/token + protected endpoint ở mức course.	Source + Postman evidence
Break	Token thiếu/sai/hết hạn; role sai; password sai; endpoint config sai.	Failure evidence
Diagnose	Đọc 401/403, log filter/authentication, token claims.	Debug note
Verify	Kiểm tra status, principal/authorities, DB/password hash, protected resource.	Security test matrix
Transfer	So sánh hand-rolled JWT với OAuth2 Resource Server/IdP production.	Architecture worksheet

0. SINH VIÊN CẦN CHUẨN BỊ GÌ TRƯỚC KHI HỌC SLOT

0.1. Kiến thức cần ôn lại

- Slot 22–23: REST CRUD, HTTP methods/status codes, JSON request/response, Controller → Service → Repository, React ↔ Spring Boot API integration.
- HTTP request: method, URL, headers, optional body; đặc biệt Authorization header và Content-Type.
- HTTP response: status, headers, body; phân biệt 400/401/403/404/409/500 ở mức ý nghĩa.
- Spring bean/DI cơ bản; request đi qua DispatcherServlet trước Controller.
- JPA entity/repository; User record có email/username, password hash, role.
- Khái niệm hash khác encryption; HTTPS/TLS bảo vệ dữ liệu trên đường truyền.

0.2. Công cụ và baseline

Thành phần	Yêu cầu	Kiểm tra nhanh
JDK	21	java -version
Maven	3.6.3+ / Maven Wrapper	mvn -v hoặc mvnw.cmd -v
IDE	IntelliJ IDEA	idea . / project build
Backend	Spring Boot 3.x baseline	GET public endpoint 200
Dependencies	Spring Web, Security, Data JPA, SQL Server, Validation, Test, JWT library nếu	Maven reload/build

Thành phần	Yêu cầu	Kiểm tra nhanh
	lab dùng	
SQL Server	DB + User table kết nối được	SSMS query
Postman	Bearer token + environment variables	public/protected smoke test
Git	Working tree sạch	git status

0.3. Pre-class 3–2–1 Learning Note

3 điều đã biết	2 điều còn mơ hồ	1 câu hỏi muốn kiểm chứng
API có endpoint public/protected; password không nên lưu plaintext; token gửi qua Authorization header.	401 khác 403 thế nào? JWT khác session ra sao?	Một request có Bearer token đi qua những filter/method nào trước Controller?

0.4. Pre-flight checklist

- Backend baseline compile/test thành công trước khi thêm Spring Security.
- Có database và ít nhất một user seed hoặc endpoint signup để tạo user.
- Không commit password DB, JWT secret hoặc private key vào Git.
- Postman collection có request public và protected.
- Bật log ở mức vừa đủ; không log password hoặc raw access token trong production.
- Commit/tag gợi ý: s24-before-security.

Baseline checkpoint

```
mvn -q -DskipTests compile
mvn test
mvn spring-boot:run
# hoặc Windows:
mvnw.cmd test
mvnw.cmd spring-boot:run
```

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

1.1. Từ CRUD API sang protected API

Trước Slot 24	Vấn đề mới	Slot 24 trả lời
Endpoint CRUD chạy được	Ai đang gọi endpoint?	Authentication xác lập identity như thế nào?
Controller/service có business rule	User được phép làm gì?	Authorization/authority/role được kiểm ở đâu?
React/Postman gọi API	Credential/token đi đâu?	Authorization header, bearer token, cookie/session
Database lưu user	Password lưu thế nào?	Adaptive one-way password hashing
HTTP status đã biết	Security failure biểu đạt thế nào?	401 vs 403; challenge/error contract
REST mong muốn stateless	Security state đặt ở đâu?	Session vs token-based model và trade-off

1.2. Mục tiêu học tập

1. Giải thích confidentiality, integrity, authentication và authorization trong REST API.
2. So sánh session-based security, HTTP Basic, Digest, certificate-based approaches và token-based security ở mức khái niệm.
3. Phân biệt OAuth 2.0 (authorization framework), OpenID Connect (identity layer – awareness) và JWT (token format).
4. Mô tả bốn vai trò OAuth 2.0: Resource Owner, Client, Authorization Server, Resource Server.
5. Giải thích JWT Header/Payload/Signature, claims, expiration và signature validation.
6. Mô tả Spring Security request lifecycle: SecurityFilterChain → authentication → SecurityContext → authorization → Controller.
7. Phân biệt 401 Unauthorized và 403 Forbidden bằng request scenario cụ thể.
8. Hiểu UserDetails/UserDetailsService, PasswordEncoder, AuthenticationProvider và AuthenticationManager ở mức course baseline.
9. Trace một login request và một protected request end-to-end.
10. Nhận biết production direction: OAuth2 Resource Server, external/central Authorization Server, asymmetric key/JWK, token rotation, BCP security.

1.3. Mental model

```
A. LOGIN / TOKEN ISSUANCE (course model)
Client
```

```
 -> POST /auth/login (email + password over HTTPS)
 -> AuthenticationController
 -> AuthenticationService
 -> AuthenticationManager / AuthenticationProvider
 -> UserDetailsService -> UserRepository -> SQL Server
 -> PasswordEncoder.matches(...)
 <- authenticated principal
 -> JwtService.generateToken(...)
 <- access token

B. PROTECTED REQUEST
Client
 -> GET /users/me
    Authorization: Bearer <token>
 -> Security Filter Chain
 -> JWT/Bearer filter
 -> extract token -> validate signature/exp -> load/map principal
 -> SecurityContext
 -> Authorization rule
 -> Controller -> Service -> Repository
 <- JSON response

C. FAILURE PATHS
No/invalid token -> 401
Valid identity but insufficient authority -> 403
```

1.4. Course-ready vs production-ready

Mức	Biểu hiện	Mục đích
Demo beginner	Hard-coded user/secret; filter chạy; endpoint protected.	Hiểu cơ chế nhưng chưa đủ an toàn.
Course-ready	User DB, password hash, clear SecurityFilterChain, JWT validation, 401/403 tests.	Đạt mục tiêu Slot 24/Lab 06.
Production direction	OAuth2/OIDC provider, Resource Server, JWK/asymmetric keys, rotation, audit/observability, RFC security BCP.	Architecture awareness; không ép lab migrate.

II. TRỌNG TÂM CỐT LÕI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

2.1. Authentication ≠ Authorization

Authentication trả lời “bạn là ai?”; authorization trả lời “bạn được phép làm gì?”. Một hệ thống có thể xác thực thành công nhưng vẫn từ chối thao tác vì thiếu authority/role. Không nên nhét business authorization rải rác bằng if/else trong controller.

2.2. Security boundary của REST API

REST endpoint là một boundary. Security phải được thực thi trước business logic: xác thực credential/token, tạo Authentication/Principal, gắn vào SecurityContext, sau đó đánh giá authorization rule.

2.3. Session-based vs stateless token

Session lưu state phía server và client mang session id; token-based thường mang credential đại diện cho authorization state ở mỗi request. Stateless không có nghĩa là “không có state ở đâu cả”: user, key, revocation, audit và policy vẫn là state của hệ thống.

2.4. HTTP Basic, Digest, certificate và custom schemes

Chapter 16 giới thiệu nhiều cơ chế để so sánh. Basic đơn giản nhưng phải đi qua HTTPS; Digest là cơ chế cũ với nonce; certificate/mTLS mạnh nhưng vận hành phức tạp; custom XAuth có rủi ro interoperability/security nếu tự thiết kế.

2.5. OAuth 2.0 – delegated authorization

OAuth 2.0 cho phép client truy cập resource thay mặt resource owner thông qua token do authorization server cấp. Bốn vai trò nền tảng: Resource Owner, Client, Authorization Server, Resource Server.

2.6. JWT – JSON Web Token

JWT là token format gồm header.payload.signature. Payload có claims nhưng không mặc định được mã hóa; ai có token có thể base64url-decode payload. Signature bảo vệ integrity/authenticity theo key/algorithm, không tạo confidentiality.

2.7. Claims và token lifecycle

Các registered claims thường gặp: sub, iss, aud, exp, nbf, iat, jti. Token validation cần xem chữ ký, thời gian hiệu lực và issuer/audience tùy contract. Expiration không thay thế logout/revocation strategy.

2.8. Spring Security filter chain

Request đi qua một chuỗi filter trước controller. SecurityFilterChain mô tả rule; authentication filter/manager/provider xác thực; SecurityContext giữ Authentication của request; authorization filter quyết định access.

2.9. UserDetails, UserDetailsService và PasswordEncoder

Course baseline có thể map User entity sang UserDetails. UserDetailsService chịu trách nhiệm load user theo username/email. PasswordEncoder dùng adaptive one-way hashing và matches() để kiểm chứng; không giải mã password.

2.10. 401 vs 403

401: request chưa được authenticated hoặc credential/token không hợp lệ; 403: identity đã authenticated nhưng không đủ quyền. Test hai trường hợp riêng để không biến mọi security failure thành 403/500.

2.11. CORS và CSRF không phải cùng một vấn đề

CORS là browser cross-origin access policy; CSRF là việc lợi dụng credential tự động gửi (thường cookie/session). Tắt CSRF chỉ vì "REST API" mà không hiểu credential model là anti-pattern.

2.12. Secret/key management

JWT signing secret/private key là credential của server. Không hard-code trong source/repository; dùng environment/secret manager. Không đưa signing secret vào React/Vite frontend.

2.13. Security decision matrix

Câu hỏi	Sai lầm người mới	Mental model đúng
JWT payload có bí mật không?	Có vì token được ký.	Không. Ký ≠ mã hóa; payload thường đọc được.
Có JWT là OAuth?	Có.	Không. JWT là format; OAuth là framework.
401 và 403 giống nhau?	Đều là "không vào được".	401: chưa/không xác thực; 403: đã xác thực nhưng thiếu quyền.
PasswordEncoder để encrypt?	Có thể decode lại.	Không; adaptive one-way hash.
JWT secret để trong React .env?	Được vì .env.	Không. VITE_* được đưa vào client bundle.
Tắt CSRF là bắt buộc cho mọi REST?	Có.	Phụ thuộc credential model; cần threat model.

III. ỨNG DỤNG THỰC TIỄN KIẾN THỨC CỦA BÀI HỌC TRONG SẢN PHẨM THỰC TẾ

3.1. Case study – Orchid Management

Use case	Public/Protected	Security concern	Evidence
POST /auth/signup	Public	validate input; hash password; duplicate email	DB hash ≠ plaintext
POST /auth/login	Public	credential verification; token issuance	200 + token / 401
GET /orchids	Có thể public	read policy	200 without token nếu contract cho phép
POST /orchids	Protected	authenticated + authority	201 with token; 401/403 negative
GET /users/me	Protected	principal identity	response reflects authenticated user
GET /users	Admin-only direction	role/authority authorization	403 for normal user

3.2. Threat-oriented thinking

Threat / lỗi	Ví dụ	Control cần nghĩ tới
Credential theft	Password/token lộ qua log hoặc HTTP	HTTPS; no secret logging; secure storage
Broken access control	User thường gọi admin endpoint	central authorization policy + tests
Token tampering	Sửa role trong payload	signature verification
Expired token	Token cũ vẫn dùng được	exp validation + clock policy
Weak password storage	plaintext/SHA-256 nhanh	BCrypt/Argon2 adaptive hash
Replay	Access token bị copy dùng lại	short lifetime; sender-constrained/token strategy direction

Threat / lỗi	Ví dụ	Control cần nghĩ tới
Key leak	JWT secret commit Git	externalized secret/key rotation

3.3. Transfer sang hệ thống production

Trong hệ thống lớn, backend business API thường đóng vai Resource Server; authentication/authorization issuance có thể được giao cho Authorization Server/Identity Provider. Spring Security hiện hỗ trợ OAuth2 Resource Server với JWT hoặc opaque bearer token. Khi dùng JWT từ Authorization Server, resource server xác minh chữ ký và các claims như issuer/expiration thay vì tự duy trì một custom login/token stack.

Lab/course model	Production direction	Lý do
App tự signup/login và ký JWT	Central Authorization Server / IdP + Resource Server	Giảm custom security code; governance/key lifecycle tốt hơn
Shared HMAC secret	Asymmetric signing + JWK/JWKS	Resource server chỉ cần public key; rotation tốt hơn
Role nhúng tùy ý	Scopes/authorities/claims contract	Interoperability + policy rõ
Manual error strings	Standard 401/403 + consistent security error contract	Client/observability tốt hơn
Token dài hạn	Short-lived access token + lifecycle strategy	Giảm blast radius

IV. NỘI DUNG CHI TIẾT – KNOWLEDGE BASE NỀN TẢNG VÀ HỆ SINH THÁI KIẾN THỨC

READING GUIDE: Phần IV là phần lý thuyết/knowledge map. Không dùng như lab checklist. Mục tiêu là tạo nền tảng đủ sâu để sinh viên có thể giải thích và tự chẩn đoán security behavior trước khi code.

4.1. Security objectives của REST API

- Confidentiality: dữ liệu nhạy cảm không bị đọc trái phép; thường dựa TLS/HTTPS và kiểm soát truy cập.
- Integrity: dữ liệu/request/token không bị sửa mà không bị phát hiện; chữ ký số/MAC và database constraints đóng vai trò khác nhau.
- Authentication: thiết lập identity đáng tin cậy của caller.
- Authorization: quyết định identity đó được phép thực hiện action nào trên resource nào.
- Accountability/Auditability: có evidence để biết ai làm gì, khi nào; nhưng log không được rò credential/token.

4.2. HTTP authentication vocabulary

- Credential: dữ liệu chứng minh danh tính/quyền, ví dụ password, client credential, bearer token.
- Principal: identity hiện tại sau khi authentication thành công.
- Authority/Role/Scope: biểu diễn permission/policy decision input.
- Challenge: server phản hồi yêu cầu client cung cấp credential, thường liên quan WWW-Authenticate.
- Bearer token: ai 'bear' token hợp lệ có thể dùng nó; vì vậy transport/storage protection rất quan trọng.

4.3. Session-based security lifecycle

- Login xác thực user → server tạo session → session id gửi trong cookie → request sau mang cookie → server lookup session.
- Ưu điểm: logout/revocation trực tiếp hơn, phù hợp server-rendered/traditional app.
- Trade-off: session store, horizontal scaling, CSRF khi browser tự gửi cookie; SPA vẫn có thể dùng secure cookie/session nếu thiết kế phù hợp.

4.4. HTTP Basic và tại sao 'Base64' không phải encryption

- Authorization: Basic <base64(username:password)> chỉ là encoding.
- Phải dùng HTTPS; nếu không credential dễ bị đọc trên đường truyền.
- Phù hợp test/internal/simple scenarios hơn là UX hiện đại; không có delegation semantics như OAuth.

4.5. Digest / certificate / custom schemes – historical & comparative awareness

- Digest từng cải thiện việc không gửi plaintext password nhưng không phải lựa chọn mặc định hiện đại.
- Certificate/mTLS có strong client/server identity nhưng quản lý PKI phức tạp.
- Custom XAuth/shared-secret protocols tăng risk vì tự thiết kế crypto/auth protocol và giảm interoperability.

- Mục tiêu học: biết trade-off, không khuyến khích tự phát minh authentication scheme.

4.6. OAuth 2.0 conceptual model

- OAuth 2.0 là authorization framework, không phải một token format và không tự mô tả user identity semantics như OpenID Connect.
- Resource Owner: chủ sở hữu resource; Client: app muốn access; Authorization Server: cấp token; Resource Server: bảo vệ API/resource.
- Access token biểu diễn delegated authorization. Refresh token (nếu có) có lifecycle và risk riêng.
- Authorization Code + PKCE là pattern hiện đại quan trọng cho public clients; implicit/password-style legacy flows cần được nhìn qua lăng kính security BCP hiện tại.

4.7. OAuth 2.0 vs OpenID Connect

- OAuth trả lời 'client có quyền gọi API nào/phạm vi nào?'.
- OIDC bổ sung identity layer trên OAuth để login/federated identity và ID Token.
- Access token không nên bị client coi mặc định như 'profile token'; ID Token và access token có audience/purpose khác nhau.

4.8. JWT anatomy

- Header thường chứa typ/alg/kid; Payload chứa claims; Signature bảo vệ integrity.
- Compact serialization: base64url(header).base64url(payload).signature.
- JWT có thể self-contained nhưng token format không tự đảm bảo revocation, encryption, audience correctness hay safe storage.

4.9. Claims

- iss: issuer; sub: subject; aud: audience; exp: expiration; nbf: not-before; iat: issued-at; jti: token id.
- Custom claims như roles cần contract rõ; không tin dữ liệu client tự gửi để tạo claims.
- Validation phải theo context: signature + expected algorithm/key + time + issuer/audience khi contract yêu cầu.

4.10. Symmetric vs asymmetric JWT signing

- HMAC (HS*) dùng shared secret cho sign/verify: đơn giản cho lab nhưng secret phải được bảo vệ ở mọi verifier.
- RSA/ECDSA dùng private key ký, public key verify: phù hợp tách Authorization Server/Resource Servers và key distribution.
- kid/JWK/JWKS hỗ trợ chọn/rotate verification key trong production ecosystems.

4.11. Spring Security architecture

- Servlet request đi qua FilterChainProxy/SecurityFilterChain trước DispatcherServlet/controller.
- AuthenticationFilter/Bearer filter tạo authentication request; AuthenticationManager/Provider xác thực; SecurityContext giữ Authentication.
- Authorization rules quyết định permit/deny. Controller nên giả định security boundary đã thực hiện policy chung, nhưng business authorization vẫn có thể cần domain rules.

4.12. AuthenticationManager / AuthenticationProvider

- AuthenticationManager nhận Authentication request object và trả Authentication đã authenticated hoặc throw exception.
- AuthenticationProvider thực hiện một strategy cụ thể, ví dụ DAO username/password.
- Provider có thể dùng UserDetailsService + PasswordEncoder để load user và verify password.

4.13. UserDetails / UserDetailsService

- UserDetails là security-facing representation của user; không bắt buộc domain entity phải trực tiếp implements nếu muốn tách concern.
- UserDetailsService.loadUserByUsername(...) là contract để load security user.
- Trong project lớn nên cân nhắc adapter để tránh domain model bị phụ thuộc security framework quá sâu.

4.14. Password storage

- Không lưu plaintext; không dùng reversible encryption làm password database strategy.
- Dùng adaptive one-way functions như BCrypt/Argon2/PBKDF2/scrypt theo ecosystem support.
- Cost phải đủ chậm để chống brute-force nhưng phù hợp capacity; có migration/rehash strategy khi policy thay đổi.

4.15. JWT authentication filter – conceptual responsibilities

- Đọc Authorization header; chỉ xử lý Bearer scheme phù hợp.
- Extract token; validate; resolve subject; load/map authorities nếu cần.
- Không overwrite authenticated SecurityContext một cách tùy tiện.
- Tiếp tục filter chain; lỗi authentication phải map đúng 401, không leak stack trace/token.

4.16. SecurityFilterChain configuration

- permitAll cho endpoint public (signup/login/docs tùy contract); authenticated cho protected API; authority-based rules cho privileged endpoints.
- Order/rule specificity phải được hiểu; cấu hình quá rộng permitAll('/**') vô hiệu security.
- CSRF, CORS, sessionCreationPolicy phải dựa threat model/credential model chứ không copy snippet mù quáng.

4.17. 401, 403 và exception translation

- Authentication failure → 401 + challenge phù hợp.
- Authorization denial sau authenticated → 403.
- AuthenticationEntryPoint và AccessDeniedHandler là extension points quan trọng để chuẩn hóa security error response.

4.18. Token expiration, logout và revocation

- JWT expiration chỉ giới hạn thời gian; token đã phát hành có thể vẫn hợp lệ tới exp nếu không có revocation mechanism.
- Logout client-side chỉ xóa token phía client; không đồng nghĩa server-side invalidation.
- Production có thể dùng short-lived access token, refresh-token rotation, denylist/introspection/opaque token tùy threat model.

4.19. CORS, CSRF, XSS và token storage

- CORS không ngăn curl/Postman; đó là browser policy.
- CSRF risk liên quan credential tự động gắn theo request (đặc biệt cookies).
- XSS có thể đánh cắp token accessible to JavaScript; vì vậy token storage decision phải đi cùng CSP/XSS threat model.
- Không có một câu trả lời 'localStorage luôn đúng' hoặc 'cookie luôn đúng' cho mọi architecture.

4.20. TLS/HTTPS

- JWT signature không bảo vệ request body/confidentiality trên đường truyền.
- HTTPS vẫn bắt buộc cho credential/token APIs.
- TLS termination, proxy headers và secure cookies là deployment concern cần tính ở production.

4.21. Authorization granularity

- URL-level rule là coarse-grained; method/domain-level rule có thể cần ownership/resource checks.
- RBAC dễ hiểu nhưng có thể quá thô; scopes/permissions/ABAC concepts giúp biểu diễn policy khác.
- Không dùng role string từ request body để tự nâng quyền.

4.22. Testing security

- Positive: valid login, valid token, right authority.
- Negative: missing token, malformed token, bad signature, expired token, wrong role, disabled user.
- Boundary tests: SecurityFilterChain/MockMvc, service authorization logic, repository/user lookup, full integration.

4.23. Observability và incident evidence

- Log authentication outcome ở mức an toàn; không log password/raw token.

- Metrics: 401/403 rate, login failures, token validation failures, latency.
- Correlation/trace id hỗ trợ điều tra nhưng không đưa sensitive claims vào log bừa bãi.

4.24. Course baseline vs current ecosystem

- Course baseline: Spring Boot 3.x, custom JWT flow theo Lab 06 để học mechanics.
- Current production direction: Spring Security OAuth2 Resource Server có first-class support cho JWT và opaque bearer tokens; Boot có auto-configuration cho issuer/JWK.
- OAuth 2.0 Security BCP (RFC 9700, 2025) cập nhật best practices; OAuth 2.1 vẫn là hướng phát triển/spec ecosystem, nên không dạy như một RFC hoàn tất nếu chưa xác minh.
- Production recommendation phải dựa official docs/RFC, không dựa snippet blog duy nhất.

V. KINH NGHIỆM THỰC TIỄN, TIPS VÀ DEBUGGING

Vấn đề	Triệu chứng	Chẩn đoán/Xử lý
401 dù token có vẻ đúng	Protected request bị reject	Kiểm Authorization scheme, signature, exp, issuer/aud, filter mapping
403 sau login	Identity có nhưng thiếu quyền	Inspect Authentication.authorities và authorization rules
Password login luôn fail	matches() false	Kiểm encoder khi signup và login có cùng strategy; DB có hash đúng
JWT payload bị sửa	Token rejected hoặc nguy hiểm nếu code không verify	Không chỉ decode; luôn verify signature/claims
Token hết hạn	401 sau một thời gian	Kiểm exp/clock; không 'fix' bằng token vô hạn
Filter không chạy	Controller vẫn nhận request hoặc 401 bất thường	Kiểm filter registration/order và SecurityFilterChain match
Endpoint public bị khóa	/auth/login trả 401	Kiểm permitAll rule trước authenticated catch-all
Endpoint admin ai cũng gọi được	Broken access control	Add authority policy + negative test
JWT secret commit Git	Security incident	Rotate secret/key; externalize secret; scrub history nếu cần
Token xuất hiện trong log	Credential leak	Mask/remove Authorization header/raw token logging
CORS lỗi nhưng Postman chạy	Browser blocked	Đây là CORS policy, không phải token validation
Disable CSRF không hiểu lý do	Có thể mở risk khi dùng cookie auth	Xác định credential model và threat model trước

5.1. Debugging decision tree

```
Request bị từ chối
|
+-- 401?
|   +-- Có Authorization header? đúng "Bearer "?
|   +-- Token parse được?
|   +-- Signature/key/algorithm đúng?
|   +-- exp/nbf/iss/aud đúng?
|   +-- User còn tồn tại/active?
|
+-- 403?
|   +-- Authentication đã tồn tại?
|   +-- authorities/scopes là gì?
|   +-- URL/method authorization rule yêu cầu gì?
|   +-- Có domain ownership rule không?
|
+-- 500?
    +-- JWT parser/library exception?
    +-- UserRepository/DB failure?
    +-- Security error bị throw qua controller thay vì entry point/handler?

Login fail
|
+-- user không tồn tại -> lookup/repository
+-- password fail -> encoder/hash
+-- account locked/disabled -> user status
+-- token generation fail -> key/claims/clock
```

5.2. Tips ở mức Slot 24

- Bắt đầu từ threat + contract, không bắt đầu từ copy một JwtAuthFilter.
- Không đặt JWT signing secret trong source, README hoặc frontend.
- Không log password, raw bearer token hoặc private key.

- Viết negative tests trước khi tuyên bố endpoint đã được bảo vệ.
- Tách 401 và 403 để client/debugger hiểu outcome.
- Dùng HTTPS trong mọi environment có credential thật.
- Course code có thể đơn giản, nhưng luôn ghi rõ điểm nào chưa production-ready.
- Khi chuyển production, ưu tiên framework-standard OAuth2/OIDC integration hơn custom auth protocol.

VI. HƯỚNG DẪN STEP BY STEP THỰC BÀI TẬP CHỦ ĐỀ VÀ BÀI TẬP TỔNG HỢP HỆ THỐNG

WORKBOOK RULE: Phần VI là workbook thực hành. Mỗi bước phải có: Mục tiêu → Thao tác → Code/config → Lệnh chạy → Verification → PASS/FAIL evidence.

6.1. Bài tập A – Security concept mapping không code

Mục tiêu: chứng minh sinh viên phân biệt được mechanism trước khi implementation.

Scenario	Sinh viên phải xác định	Kết quả cần nộp
Browser login truyền thống	session/cookie/CSRF implications	sequence diagram
SPA gọi REST API	token/cookie choices + CORS	threat worksheet
Third-party app gọi API thay user	OAuth roles	role mapping diagram
JWT bị sửa payload role=ADMIN	integrity/signature	explanation + expected 401

6.2. Bài tập B – Tạo micro-project security baseline

Mục tiêu: tạo project Spring Boot security nhỏ để quan sát filter chain và authentication.

Windows/CMD workflow

```
mkdir sba301-slot24-security
cd sba301-slot24-security
# Tạo project bằng Spring Initializr (IntelliJ hoặc start.spring.io)
# Dependencies:
# Spring Web
# Spring Security
# Spring Data JPA
# Validation
# MS SQL Server Driver
# Spring Boot Starter Test
idea .
```

Verification: project mở trong IntelliJ, Maven dependencies resolve xanh, `mvn test` PASS.

6.3. Bài tập C – Cấu hình datasource và User entity

application.properties

```
spring.datasource.url=jdbc:sqlserver://localhost:1433;databaseName=OrchidManagementDB;encrypt=true;trustServer
Certificate=true
spring.datasource.username=sa
spring.datasource.password=${DB_PASSWORD}
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# KHÔNG commit DB_PASSWORD thật.
```

Entity baseline

```
@Entity
@Table(name = "AppUser")
public class AppUser {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String email;
```

```
    @Column(nullable = false)
    private String passwordHash;

    @Column(nullable = false)
    private String role = "USER";

    private boolean enabled = true;

    // getters/setters
}
```

Repository

```
public interface AppUserRepository extends JpaRepository<AppUser, Long> {
    Optional<AppUser> findByEmail(String email);
    boolean existsByEmail(String email);
}
```

Verification SQL:

```
SELECT TOP (100) id, email, password_hash, role, enabled
FROM dbo.AppUser
ORDER BY id DESC;
```

CHECKPOINT: PASS: bảng tồn tại; password chưa cần có dữ liệu. FAIL: datasource/DDL error → đọc startup log trước khi làm security.

6.4. Bài tập D – PasswordEncoder và signup

Password encoder

```
@Bean
PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Request DTO

```
public record SignupRequest(
    @Email @NotBlank String email,
    @Size(min = 8) String password
) {}
```

Signup service core

```
@Service
public class AuthService {
    private final AppUserRepository users;
    private final PasswordEncoder encoder;

    public AuthService(AppUserRepository users, PasswordEncoder encoder) {
        this.users = users;
        this.encoder = encoder;
    }

    @Transactional
    public AppUser signup(SignupRequest req) {
        if (users.existsByEmail(req.email())) {
            throw new IllegalStateException("EMAIL_EXISTS");
        }
        AppUser user = new AppUser();
        user.setEmail(req.email());
        user.setPasswordHash(encoder.encode(req.password()));
        user.setRole("USER");
        return users.save(user);
    }
}
```

Postman request

```
POST http://localhost:8080/auth/signup
Content-Type: application/json

{
  "email": "student24@example.com",
  "password": "Sba301@2026"
}
```

Verification: DB phải chứa hash bắt đầu dạng bcrypt thường gặp `$2...`, tuyệt đối không bằng `Sba301@2026`.

CHECKPOINT: PASS: create user + password hash. FAIL nếu DB lưu plaintext.

6.5. Bài tập E – UserDetailsService / authentication provider

Course-level security beans

```
@Bean
UserDetailsService userDetailsService(AppUserRepository users) {
    return username -> users.findByEmail(username)
        .map(u -> User.withUsername(u.getEmail())
            .password(u.getPasswordHash())
            .roles(u.getRole())
            .disabled(!u.isEnabled())
            .build())
        .orElseThrow(() -> new UsernameNotFoundException(username));
}

@Bean
AuthenticationProvider authenticationProvider(
        UserDetailsService uds, PasswordEncoder encoder) {
    DaoAuthenticationProvider provider = new DaoAuthenticationProvider(uds);
    provider.setPasswordEncoder(encoder);
    return provider;
}

@Bean
AuthenticationManager authenticationManager(
        AuthenticationConfiguration config) throws Exception {
    return config.getAuthenticationManager();
}
```

VERSION RULE: Tùy Spring Security minor version, constructor/setter signatures có thể khác. Sinh viên phải compile theo version project và đối chiếu official API thay vì chép mù.

6.6. Bài tập F – SecurityFilterChain trước JWT

Baseline rules

```
@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.disable()) // chỉ cho lab bearer-token stateless; hiểu threat model
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/auth/**").permitAll()
            .requestMatchers(HttpMethod.GET, "/orchids/**").permitAll()
            .anyRequest().authenticated()
        )
        .sessionManagement(sm -> sm
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
        );
    return http.build();
}
```

Verification trước JWT filter:

```
GET /orchids             -> 200 nếu public rule
POST /orchids            -> 401
POST /auth/signup        -> permitAll
GET /users/me            -> 401
```

CHECKPOINT: PASS: public/protected boundary đúng. Đây là checkpoint quan trọng trước khi thêm token.

6.7. Bài tập G – JWT service (course mechanics)

Nếu Lab 06 dùng `io.jsonwebtoken` (JJWT), tạo service với secret externalized. Đoạn dưới chỉ minh họa responsibilities; version API có thể khác.

JWT service skeleton

```
@Value("${security.jwt.secret}")
private String secret;

@Value("${security.jwt.expiration-ms:900000}")
private long expirationMs;

public String generateToken(UserDetails user) {
    Instant now = Instant.now();
    return Jwts.builder()
        .subject(user.getUsername())
        .issuedAt(Date.from(now))
        .expiration(Date.from(now.plusMillis(expirationMs)))
        .signWith(getSigningKey())
        .compact();
}

public String extractSubject(String token) {
    return parseClaims(token).getSubject();
}

public boolean isValid(String token, UserDetails user) {
    Claims claims = parseClaims(token);
    return claims.getSubject().equals(user.getUsername())
        && claims.getExpiration().after(new Date());
}
```

Externalized secret

```
# Ví dụ lab local – dùng secret đủ dài và KHÔNG commit:
security.jwt.secret=${JWT_SECRET}
security.jwt.expiration-ms=900000
```

SECURITY: Không đưa `JWT_SECRET` vào React `.env`. VITE_* là dữ liệu client-visible.

6.8. Bài tập H – Login và token issuance

Login service

```
public record LoginRequest(String email, String password) {}
public record LoginResponse(String accessToken, String tokenType) {}

public LoginResponse login(LoginRequest req) {
    Authentication auth = authenticationManager.authenticate(
        new UsernamePasswordAuthenticationToken(
            req.email(), req.password())
    );

    UserDetails principal = (UserDetails) auth.getPrincipal();
    String token = jwtService.generateToken(principal);
    return new LoginResponse(token, "Bearer");
}
```

Authentication controller

```
@RestController
@RequestMapping("/auth")
public class AuthController {
    private final AuthService authService;

    @PostMapping("/login")
    public ResponseEntity<LoginResponse> login(
            @RequestBody LoginRequest request) {
        return ResponseEntity.ok(authService.login(request));
    }
}
```

Postman

```
POST http://localhost:8080/auth/login
Content-Type: application/json

{
  "email": "student24@example.com",
  "password": "Sba301@2026"
}
```

Verification: đúng password → 200 + token; sai password → 401 (không phải 500). Decode payload chỉ để quan sát claims, không xem decode là validation.

6.9. Bài tập I – JWT authentication filter

Conceptual filter implementation

```
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {
    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    public JwtAuthenticationFilter(
            JwtService jwtService,
            UserDetailsService userDetailsService) {
        this.jwtService = jwtService;
        this.userDetailsService = userDetailsService;
    }

    @Override
    protected void doFilterInternal(
            HttpServletRequest request,
            HttpServletResponse response,
            FilterChain filterChain)
            throws ServletException, IOException {

        String header = request.getHeader("Authorization");

        if (header == null || !header.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        String token = header.substring(7);
        String username = jwtService.extractSubject(token);

        if (username != null &&
            SecurityContextHolder.getContext().getAuthentication() == null) {

            UserDetails user = userDetailsService.loadUserByUsername(username);

            if (jwtService.isValid(token, user)) {
```

```
                UsernamePasswordAuthenticationToken auth =
                    new UsernamePasswordAuthenticationToken(
                        user, null, user.getAuthorities());

                auth.setDetails(
                    new WebAuthenticationDetailsSource().buildDetails(request));

                SecurityContextHolder.getContext().setAuthentication(auth);
            }
        }

        filterChain.doFilter(request, response);
    }
}
```

Đăng ký filter trước `UsernamePasswordAuthenticationFilter`:

```
http.addFilterBefore(
    jwtAuthenticationFilter,
    UsernamePasswordAuthenticationFilter.class
);
```

SECURITY: Production code phải xử lý parser exceptions/invalid token rõ ràng và không để malformed token biến thành 500.

6.10. Bài tập J – Protected endpoint và 401/403

Protected principal endpoint

```
@GetMapping("/users/me")
public Map<String, Object> me(Authentication authentication) {
    return Map.of(
        "name", authentication.getName(),
        "authorities", authentication.getAuthorities()
    );
}
```

Postman

```
GET http://localhost:8080/users/me
Authorization: Bearer {{access_token}}
```

Case	Expected	Evidence
Không token	401	status + WWW-Authenticate nếu framework trả
Token hợp lệ	200	principal + authorities
Token bị sửa 1 ký tự	401	validation failure
Token hết hạn	401	expiration failure
Authenticated USER gọi ADMIN endpoint	403	authorization denied

6.11. Bài tập K – Role/authority rule

```
http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/auth/**").permitAll()
    .requestMatchers(HttpMethod.GET, "/orchids/**").permitAll()
    .requestMatchers("/admin/**").hasRole("ADMIN")
    .anyRequest().authenticated()
);
```

Tạo user USER và ADMIN. Test đủ: ADMIN→200, USER→403, no token→401.

6.12. Bài tập L – Break-It → Diagnose → Fix

Break	Kỳ vọng	Điểm chẩn đoán
Bỏ 'Bearer ' prefix	401	Authorization header parser
Sửa 1 ký tự payload/signature	401	signature validation
Set expiration 1 giây	401 sau expiry	exp + clock
PasswordEncoder signup/login khác nhau	login fail	hash + matches
permitAll('/**')	protected endpoint 200 không token	authorization config review
ROLE_USER nhưng rule hasRole('ADMIN')	403	authorities
Throw JWT parser exception không catch	500	filter error mapping

6.13. Bài tập M – Trace bằng debugger

Đặt breakpoint tối thiểu tại:

- AuthController.login(...)
- AuthenticationManager.authenticate(...) call site
- UserDetailsService.loadUserByUsername(...)
- PasswordEncoder.matches(...) call path (quan sát qua debugger/call stack)
- JwtService.generateToken(...)
- JwtAuthenticationFilter.doFilterInternal(...)
- JwtService validation/parse method
- protected Controller method

Trace worksheet

```
LOGIN:
POST /auth/login
 -> AuthController.login
 -> AuthService.login
 -> AuthenticationManager
 -> AuthenticationProvider
 -> UserDetailsService
 -> PasswordEncoder.matches
 <- authenticated principal
 -> JwtService.generateToken
 <- token

PROTECTED:
GET /users/me + Bearer
 -> SecurityFilterChain
 -> JwtAuthenticationFilter
 -> JwtService.parse/validate
 -> UserDetailsService
 -> SecurityContext
 -> authorization
 -> Controller
```

6.14. Bài tập N – Automated security tests

Minimum regression suite

```
@SpringBootTest
@AutoConfigureMockMvc
class SecurityIntegrationTest {

    @Autowired MockMvc mvc;

    @Test
    void meWithoutToken_returns401() throws Exception {
        mvc.perform(get("/users/me"))
           .andExpect(status().isUnauthorized());
    }

    // Bổ sung:
    // login valid -> 200
    // login bad password -> 401
    // protected with token -> 200
    // admin endpoint with USER -> 403
}
```

6.15. Integration Gate G1–G12

Gate	Bằng chứng bắt buộc
G1 Build	compile/test pass

Gate	Bằng chứng bắt buộc
G2 DB/User	User table + password hash
G3 Public boundary	/auth/** permit; public endpoint đúng contract
G4 Protected boundary	protected endpoint 401 without token
G5 Login	valid 200; invalid credential 401
G6 JWT	signature + expiration validation
G7 Principal	/users/me reflects authenticated identity
G8 Authorization	USER/ADMIN 403/200 cases
G9 Negative token	malformed/tampered/expired rejected
G10 Trace	login + protected request method chain
G11 Break/Fix	ít nhất 2 security failures được chẩn đoán
G12 Explain	human verification: JWT ≠ OAuth; 401 ≠ 403

6.16. Bài tập tổng hợp hệ thống

1. Hoàn thiện signup/login với password hashing.
2. Cấu hình public và protected endpoints.
3. Tạo access token có subject + expiration.
4. Bảo vệ ít nhất POST/PUT/DELETE Orchid.
5. Tạo endpoint `/users/me`.
6. Tạo ít nhất một endpoint ADMIN-only.
7. Chạy test matrix positive + negative.
8. Trace login và protected request bằng debugger/log.
9. Viết security README: threat assumptions, endpoints, token lifetime, known limitations.
10. Viết production migration note: custom JWT lab → OAuth2 Resource Server/Authorization Server.

VII. TỪ ĐIỂN THUẬT NGỮ – FOUNDATION GLOSSARY

Thuật ngữ	Định nghĩa ngắn	Ví dụ Slot 24
Authentication	Xác lập identity của caller	username/password → Authentication
Authorization	Quyết định quyền truy cập	hasRole('ADMIN')
Principal	Identity hiện tại	authentication.getName()
GrantedAuthority	Quyền/role đã cấp	ROLE_USER
SecurityContext	Nơi giữ Authentication của request/thread context	SecurityContextHolder
SecurityFilterChain	Chuỗi filter/rule security cho request	HttpSecurity.build()
AuthenticationManager	Điều phối authentication	authenticate(...)
AuthenticationProvider	Strategy xác thực cụ thể	DaoAuthenticationProvider
UserDetails	Security-facing user representation	username/password/authorities
UserDetailsService	Load user cho authentication	loadUserByUsername
PasswordEncoder	Adaptive password hashing contract	BCryptPasswordEncoder
Bearer Token	Token được gửi như credential bearer	Authorization: Bearer ...
JWT	JSON Web Token	header.payload.signature
Claim	Thuộc tính trong JWT payload	sub, exp, iss, aud
Signature	Bảo vệ integrity/authenticity của JWT	HMAC/RSA/ECDSA
OAuth 2.0	Delegated authorization framework	client/resource server
Resource Owner	Chủ sở hữu resource	end user
Client	Application yêu cầu access	SPA/mobile/server app
Authorization Server	Cấp token	IdP/Auth server
Resource Server	API bảo vệ resource	Spring Boot business API
Access Token	Credential truy cập resource	JWT hoặc opaque
Refresh Token	Credential xin access token mới	server/client lifecycle
Scope	Phạm vi quyền OAuth	messages.read
401 Unauthorized	Chưa/không authentication hợp lệ	missing/invalid token
403 Forbidden	Authenticated nhưng thiếu quyền	USER gọi ADMIN endpoint
CSRF	Cross-Site Request Forgery	cookie-based credential abuse
CORS	Browser cross-origin policy	localhost:5173 → :8080
JWK/JWKS	JSON Web Key / key set	public verification keys
OIDC	Identity layer trên OAuth 2.0	ID Token/login federation
PKCE	Proof Key for Code Exchange	public client authorization code flow

VIII. RESEARCH BASE VÀ AI-BASED LEARNING – 4 NHÓM TRẢI NGHIỆM CUỐI BUỔI

8.1. Quy tắc sử dụng AI

- AI không được tự bịa class/filter/method không có trong source.
- Claim về execution flow phải kiểm bằng debugger, stack trace, source navigation hoặc logs.
- Claim về cryptography/security standard phải kiểm bằng official docs/RFC.

- Không paste password, JWT secret, private key hoặc token thật vào prompt.
- Phân biệt rõ Course Baseline và Current Production Direction.
- Output AI là hypothesis/learning aid; evidence cuối cùng thuộc về sinh viên.

8.2. NHÓM I – Phân tích workflow và quy luật thực thi

PROMPT I.1: Phân tích startup flow của Spring Security từ SpringApplication.run đến khi SecurityFilterChain sẵn sàng. Chỉ dùng class/bean có trong source tôi cung cấp; chỉ rõ bean nào được tạo và dependency graph.

PROMPT I.2: Trace POST /auth/login từ HTTP request đến AuthenticationManager, AuthenticationProvider, UserDetailsService, PasswordEncoder và JwtService. Với mỗi bước ghi input/output và breakpoint nên đặt.

PROMPT I.3: Trace GET /users/me có Bearer token từ servlet filter chain đến Controller. Phân biệt framework method với application method.

PROMPT I.4: So sánh flow khi request không có token, token malformed, token expired và token hợp lệ. Dự đoán status rồi yêu cầu tôi kiểm chứng bằng Postman.

PROMPT I.5: Giải thích SecurityContext được tạo/điền/đọc khi nào trong một protected request của project.

PROMPT I.6: Phân tích vì sao filter phải gọi filterChain.doFilter và chuyện gì xảy ra nếu return sai chỗ.

PROMPT I.7: Từ source SecurityFilterChain của tôi, lập bảng URL → rule → expected 200/401/403.

PROMPT I.8: Trace signup: DTO validation → passwordEncoder.encode → repository.save → SQL INSERT; xác định nơi không được log password.

PROMPT I.9: Vẽ sequence diagram login và protected request; đánh dấu trust boundary, credential và sensitive data.

PROMPT I.10: Tạo 12 câu hỏi source/log-driven để tôi tự chứng minh security flow thay vì học thuộc annotation.

8.3. NHÓM II – 10 prompt kỹ thuật nâng cao

PROMPT II.1: Phân tích chi tiết JWT signature validation: signing input là gì, base64url là gì, vì sao decode payload không phải verify. Dùng ví dụ token giả và chỉ ra tampering experiment an toàn.

PROMPT II.2: So sánh HS256 và RS256/ES256 về key ownership, distribution, rotation, blast radius và deployment topology. Kết luận khi nào lab dùng HMAC, production dùng asymmetric hợp lý.

PROMPT II.3: Phân tích registered claims iss/sub/aud/exp/nbf/iat/jti. Thiết kế validation matrix và các negative test tương ứng cho Spring Boot Resource Server.

PROMPT II.4: Giải thích JWT revocation problem. So sánh short-lived access token, denylist, opaque token introspection, refresh token rotation và sender-constrained token.

PROMPT II.5: So sánh RBAC, permission-based authorization, OAuth scopes và ABAC cho FUNewsManagement/Orchid Management. Thiết kế policy matrix.

PROMPT II.6: Phân tích CSRF trong ba kiến trúc: session cookie, JWT trong Authorization header, JWT trong HttpOnly cookie. Không đưa kết luận một-size-fits-all.

PROMPT II.7: Phân tích XSS và token storage cho SPA: memory, localStorage, sessionStorage, HttpOnly cookie. Lập threat/trade-off table; không chỉ liệt kê best practice.

PROMPT II.8: Giải thích key rotation với kid + JWKS: Authorization Server publish keys, Resource Server cache/refresh key như thế nào; đề xuất test rotation.

PROMPT II.9: Phân tích clock skew và token time claims; thiết kế test expired/not-before với clock có thể kiểm soát.

PROMPT II.10: Review một custom JwtAuthenticationFilter và tìm 15 lỗi tiềm ẩn: exception mapping, algorithm confusion, missing issuer/audience, logging token, duplicate user lookup, context overwrite, etc.

8.4. NHÓM III – 10 prompt production engineering

PROMPT III.1: Đóng vai Security Architect, review kiến trúc custom signup/login/JWT của lab và đề xuất migration sang Authorization Server/IdP + Spring OAuth2 Resource Server. Giữ mental model nhưng giảm custom code.

PROMPT III.2: Thiết kế security architecture cho e-commerce gồm SPA, API Gateway, Product, Order, Payment. Xác định token audience/scope, service-to-service auth và trust boundaries.

PROMPT III.3: Thiết kế secret/key management lifecycle: dev/staging/prod, secret manager, rotation, compromise response và CI/CD exposure prevention.

PROMPT III.4: Thiết kế standardized security error contract cho 401/403 nhưng vẫn tuân thủ WWW-Authenticate và không leak chi tiết authentication failure.

PROMPT III.5: Xây logging/metrics/tracing policy cho authentication: log gì, không log gì, correlation id, anomaly metrics, alert thresholds.

PROMPT III.6: Thiết kế rate limiting / brute-force defense cho login: per account, per IP, lockout trade-off, CAPTCHA/risk signal direction; phân tích denial-of-service risk.

PROMPT III.7: Thiết kế audit trail cho admin actions: subject, action, resource, result, timestamp, traceId; phân biệt audit log với application debug log.

PROMPT III.8: Thiết kế regression security test suite: MockMvc/WebTest, testcontainers/DB, token fixtures, expired/tampered/role cases và CI gate.

PROMPT III.9: Phân tích multi-tenant authorization: tenant claim, resource ownership, repository query filtering và broken object-level authorization risk.

PROMPT III.10: Threat-model endpoint POST /orders hoặc POST /payments: authentication, authorization, replay/idempotency, data integrity, observability và incident response.

8.5. NHÓM IV – 10 prompt current ecosystem / community trends

PROMPT IV.1: Kiểm tra official Spring Security docs hiện tại: phiên bản stable, thay đổi cấu hình SecurityFilterChain so với Spring Security 6 course baseline, deprecated APIs và migration direction. Ghi URL + ngày truy cập.

PROMPT IV.2: Khảo sát official Spring Security OAuth2 Resource Server support cho JWT và opaque token. So sánh dependencies, issuer-uri, jwk-set-uri, audience validation và scope mapping.

PROMPT IV.3: Kiểm tra Spring Authorization Server/OAuth2 Authorization Server hiện tại: supported grants/token formats/PKCE/DPoP. Phân loại MUST KNOW / SHOULD EXPLORE / WATCH LATER.

PROMPT IV.4: Đọc RFC 9700 OAuth 2.0 Security Best Current Practice và tóm tắt 10 khuyến nghị ảnh hưởng trực tiếp đến SPA/API. Chỉ ra flow/anti-pattern nào không nên dạy như mặc định hiện đại.

PROMPT IV.5: Xác minh trạng thái OAuth 2.1 hiện tại từ IETF. Không gọi nó là RFC nếu chưa phải RFC; chỉ ra nội dung security nào đã được đưa từ BCP vào hướng OAuth 2.1.

PROMPT IV.6: Khảo sát password hashing recommendation hiện tại trong Spring Security: BCrypt, Argon2, DelegatingPasswordEncoder; cách tune cost và migration hash.

PROMPT IV.7: Khảo sát DPoP/sender-constrained access tokens trong ecosystem Spring Security hiện tại: vấn đề token replay nào nó giải quyết và maturity/adoption direction.

PROMPT IV.8: So sánh self-contained JWT và opaque access token trong production hiện tại: latency, revocation, privacy, key management, introspection dependency.

PROMPT IV.9: Khảo sát Passkeys/WebAuthn và MFA trong modern authentication ecosystem. Giải thích chúng liên quan thế nào đến login nhưng không thay thế authorization token model.

PROMPT IV.10: Tạo '2026 Security Technology Radar' cho Spring Boot REST: password auth, OAuth/OIDC, Resource Server, Authorization Server, JWT/JWK, opaque token, DPoP, mTLS, passkeys, secret management, observability; mỗi mục có evidence từ nguồn chính thức.

8.6. AI Verification Worksheet

Prompt/claim AI	Nguồn kiểm chứng	Thử nghiệm	Kết luận SV
JWT filter execution order	Source + debugger + Spring docs	breakpoint filter/controller	
401 vs 403 behavior	Security tests/Postman	missing token vs wrong role	
Password hashing recommendation	Spring Security official docs	encode/matches benchmark	
OAuth/JWT claim	RFC/Spring official docs	architecture comparison	
Current version/deprecation	official release/reference docs	compile sample	

IX. SINH VIÊN CẦN TỰ KIỂM TRA GÌ SAU BUỔI HỌC

□ 1. Tôi có thể giải thích authentication và authorization bằng ví dụ khác nhau.

□ 2. Tôi có thể giải thích vì sao JWT không đồng nghĩa OAuth 2.0.

□ 3. Tôi có thể nêu bốn vai trò OAuth 2.0 và đặt Orchid API vào vai Resource Server khi phù hợp.

□ 4. Tôi có thể giải thích header/payload/signature và vì sao JWT payload không phải bí mật.

□ 5. Tôi có thể giải thích sub/iss/aud/exp/nbf/iat và validation cần thiết.

□ 6. Tôi có thể trace login flow qua AuthenticationManager/Provider/UserDetailsService/PasswordEncoder.

□ 7. Tôi có thể trace protected request qua SecurityFilterChain/JWT filter/SecurityContext đến Controller.

□ 8. Tôi phân biệt chính xác 401 và 403.

□ 9. Tôi biết tại sao password phải hash bằng adaptive one-way function.

□ 10. Tôi biết vì sao JWT signing secret/private key không được nằm trong frontend/source control.

□ 11. Tôi có thể tạo negative tests cho missing/tampered/expired/wrong-role token.

□ 12. Tôi hiểu CORS, CSRF và XSS là ba vấn đề khác nhau.

□ 13. Tôi có thể mô tả limitation của custom JWT lab và production migration direction.

□ 14. Tôi có evidence Postman + DB + debugger/log cho ít nhất một login và một protected request.

□ 15. Tôi có thể trả lời 'tại sao hệ thống an toàn hơn' bằng evidence/threat model thay vì chỉ nói 'có Spring Security/JWT'.

IX.1. Exit ticket 5 phút

Câu hỏi	Tự trả lời ngắn
JWT được ký có nghĩa payload được mã hóa không?	
Request không token và request USER vào ADMIN endpoint khác status gì?	
OAuth 2.0 bốn roles là gì?	
PasswordEncoder.matches làm gì?	
Một claim/security recommendation hôm nay bạn đã tự kiểm bằng evidence nào?	

TÀI LIỆU THAM CHIẾU & SOURCE TRACEABILITY

- Course source: Chapter 16 – Securing REST Services with Spring Boot. Nội dung gốc: session-based security, HTTP Basic, Digest, certificate-based, XAuth, OAuth và JWT implementation.
- Course source: Lab 06 – Secure REST Services with Spring Boot using JSON Web Token (JWT). Baseline: Spring Boot 3 + SQL Server + Spring Data JPA; registration/login; UserDetails; JWT classes; security configuration; filter; auth controller; protected endpoints.
- Official: Spring Security Reference – OAuth 2.0 Resource Server (JWT / opaque bearer tokens).
- Official: Spring Boot Reference – OAuth2 client/resource server auto-configuration.
- Official: Spring Security Reference – Password Storage (BCrypt/Argon2 and adaptive one-way functions).
- Standard: RFC 7519 – JSON Web Token (JWT).
- Security BCP: RFC 9700 (January 2025) – Best Current Practice for OAuth 2.0 Security.

NOTE: Course Baseline vs Current Production Radar: tài liệu giữ Spring Boot 3.x/Spring Security 6.x style của course khi viết lab. Các ví dụ Spring Security 7.x/OAuth2 Resource Server, DPoP hoặc Authorization Server được trình bày như production/current-ecosystem awareness và phải kiểm lại official docs theo ngày học.